

lab15(SSU)

October 26, 2025

1 Lab Assignment 15: Testing Hypotheses (2)

Please complete this notebook by filling in the cells provided. Before you begin, execute the previous cell to load the provided tests.

Helpful Resource:

- [Python Reference](#): Cheat sheet of helpful array & table methods!

Recommended Readings:

- [Sampling Methods Guide](#)
- [Testing Hypotheses](#)

```
[27]: !apt-get install texlive texlive-xetex texlive-latex-extra pandoc
      !pip install py pandoc
      !pip install datascience
```

```
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
pandoc is already the newest version (2.9.2.1-3ubuntu2).
texlive is already the newest version (2021.20220204-1).
texlive-latex-extra is already the newest version (2021.20220204-1).
texlive-xetex is already the newest version (2021.20220204-1).
0 upgraded, 0 newly installed, 0 to remove and 38 not upgraded.
Requirement already satisfied: py pandoc in /usr/local/lib/python3.12/dist-packages (1.15)
Requirement already satisfied: datascience in /usr/local/lib/python3.12/dist-packages (0.18.0)
Requirement already satisfied: folium>=0.9.1 in /usr/local/lib/python3.12/dist-packages (from datascience) (0.20.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from datascience) (75.2.0)
Requirement already satisfied: matplotlib>=3.0.0 in /usr/local/lib/python3.12/dist-packages (from datascience) (3.10.0)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from datascience) (2.2.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from datascience) (1.16.2)
```

Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from datascience) (2.0.2)

Requirement already satisfied: ipython in /usr/local/lib/python3.12/dist-packages (from datascience) (7.34.0)

Requirement already satisfied: plotly in /usr/local/lib/python3.12/dist-packages (from datascience) (5.24.1)

Requirement already satisfied: branca in /usr/local/lib/python3.12/dist-packages (from datascience) (0.8.2)

Requirement already satisfied: jinja2>=2.9 in /usr/local/lib/python3.12/dist-packages (from folium>=0.9.1->datascience) (3.1.6)

Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from folium>=0.9.1->datascience) (2.32.4)

Requirement already satisfied: xyzservices in /usr/local/lib/python3.12/dist-packages (from folium>=0.9.1->datascience) (2025.4.0)

Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0.0->datascience) (1.3.3)

Requirement already satisfied: cycycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0.0->datascience) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0.0->datascience) (4.60.1)

Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0.0->datascience) (1.4.9)

Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0.0->datascience) (25.0)

Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0.0->datascience) (11.3.0)

Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0.0->datascience) (3.2.5)

Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0.0->datascience) (2.9.0.post0)

Requirement already satisfied: jedi>=0.16 in /usr/local/lib/python3.12/dist-packages (from ipython->datascience) (0.19.2)

Requirement already satisfied: decorator in /usr/local/lib/python3.12/dist-packages (from ipython->datascience) (4.4.2)

Requirement already satisfied: pickleshare in /usr/local/lib/python3.12/dist-packages (from ipython->datascience) (0.7.5)

Requirement already satisfied: traitlets>=4.2 in /usr/local/lib/python3.12/dist-packages (from ipython->datascience) (5.7.1)

Requirement already satisfied: prompt-toolkit!=3.0.0,!3.0.1,<3.1.0,>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from ipython->datascience) (3.0.52)

Requirement already satisfied: pygments in /usr/local/lib/python3.12/dist-packages (from ipython->datascience) (2.19.2)

Requirement already satisfied: backcall in /usr/local/lib/python3.12/dist-packages (from ipython->datascience) (0.2.0)

Requirement already satisfied: matplotlib-inline in /usr/local/lib/python3.12/dist-packages (from ipython->datascience) (0.1.7)

Requirement already satisfied: pexpect>4.3 in /usr/local/lib/python3.12/dist-packages (from ipython->datascience) (4.9.0)

Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->datascience) (2025.2)

Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->datascience) (2025.2)

Requirement already satisfied: tenacity>=6.2.0 in /usr/local/lib/python3.12/dist-packages (from plotly->datascience) (8.5.0)

Requirement already satisfied: parso<0.9.0,>=0.8.4 in /usr/local/lib/python3.12/dist-packages (from jedi>=0.16->ipython->datascience) (0.8.5)

Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2>=2.9->folium>=0.9.1->datascience) (3.0.3)

Requirement already satisfied: ptyprocess>=0.5 in /usr/local/lib/python3.12/dist-packages (from pexpect>4.3->ipython->datascience) (0.7.0)

Requirement already satisfied: wcwidth in /usr/local/lib/python3.12/dist-packages (from prompt-toolkit!=3.0.0,!3.0.1,<3.1.0,>=2.0.0->ipython->datascience) (0.2.14)

Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib>=3.0.0->datascience) (1.17.0)

Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->folium>=0.9.1->datascience) (3.4.4)

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->folium>=0.9.1->datascience) (3.11)

Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->folium>=0.9.1->datascience) (2.5.0)

Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->folium>=0.9.1->datascience) (2025.10.5)

```
[28]: # Connect Google Drive to Colab so you can access your files
from google.colab import drive
drive.mount('/content/drive')

import os
os.chdir('/content/drive/MyDrive/Colab Notebooks/')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
[29]: # Run this cell to set up the notebook, but please don't change it.
```

```
# These lines import the Numpy and Datascience modules.
```

```
import numpy as np
```

```
from datascience import *
```

```
# These lines do some fancy plotting magic.
```

```
import matplotlib
```

```
%matplotlib inline
```

```
import matplotlib.pyplot as plt
```

```
plt.style.use('fivethirtyeight')
```

```
import warnings
```

```
warnings.simplefilter('ignore', FutureWarning)
```

1.1 1. Who is Older?

Data scientists have drawn a simple random sample of size 500 from a large population of adults. Each member of the population happened to identify as either “male” or “female”. (Though many people identify outside of the gender binary, in this particular population of interest, each member happened to identify as either male or female.) Data was collected on several attributes of the sampled people, including age. The table `sampled_ages` contains one row for each person in the sample, with columns containing the individual’s gender identity.

```
[30]: sampled_ages = Table.read_table('./DS/age.csv')
      sampled_ages.show(5)
```

<IPython.core.display.HTML object>

Question 1.1. How many females were there in our sample? Please use the provided skeleton code. (4 points)

Hint: Keep in mind that `.group` sorts categories in alphabetical order!

```
[31]: num_females = sampled_ages.group('Gender').where('Gender', 'female').
      ↪column('count').item(0)
      num_females
```

```
[31]: 260
```

Question 1.2. Complete the cell below so that `avg_male_vs_female` evaluates to `True` if the sampled males are older than the sampled females on average, and `False` otherwise. Use Python code to achieve this. (4 points)

```
[32]: group_mean_tbl = sampled_ages.group('Gender', np.mean)
      group_means = group_mean_tbl.column('Age mean')           # array of mean ages
      avg_male_vs_female = group_means.item(1) > group_means.item(0)
      avg_male_vs_female
```

```
[32]: True
```

Question 1.3. The data scientists want to use the data to test whether males are older than females—or, in other words, whether the ages of the two groups have the same distribution. One of the following statements is their null hypothesis and another is their alternative hypothesis. Assign `null_statement_number` and `alternative_statement_number` to the numbers of the correct statements in the code cell below. (4 points)

1. In the sample, the males and females have the same distribution of ages; the sample averages of the two groups are different due to chance.
2. In the population, the males and females have the same distribution of ages; the sample averages of the two groups are different due to chance.
3. The age distributions of males and females in the population are different due to chance.
4. The males in the sample are older than the females, on average.
5. The males in the population are older than the females, on average.
6. The average ages of the males and females in the population are different.

```
[33]: null_statement_number = 2
      alternative_statement_number = 5
```

Question 1.4. The data scientists have decided to use a permutation test. Assign `permutation_test_reason` to the number corresponding to the reason they made this choice. (4 points)

1. Since a person's age shouldn't be related to their gender, it doesn't matter who is labeled "male" and who is labeled "female", so you can use permutations.
2. Under the null hypothesis, permuting the labels in the `sampled_ages` table is equivalent to drawing a new random sample with the same number of males and females as in the original sample.
3. Under the null hypothesis, permuting the rows of `sampled_ages` table is equivalent to drawing a new random sample with the same number of males and females as in the original sample.

Note: Check out [12.1](#) for a refresher on random permutations and permutation tests.

```
[34]: permutation_test_reason = 2
      permutation_test_reason
```

```
[34]: 2
```

Question 1.5. To test their hypotheses, the data scientists have followed our textbook's advice and chosen a test statistic where the following statement is true: Large values of the test statistic favor the alternative hypothesis.

The data scientists' test statistic is one of the two options below. Which one is it? Assign the appropriate number to the variable `correct_test_stat`. (4 points)

1. "male age average - female age average" in a sample created by randomly shuffling the male/female labels
2. "|male age average - female age average|" in a sample created by randomly shuffling the male/female labels

```
[35]: correct_test_stat = 1
      correct_test_stat
```

[35]: 1

Question 1.6. Complete the cell below so that `observed_statistic_ab` evaluates to the observed value of the data scientists' test statistic. Use as many lines of code as you need, and remember that you can use any quantity, table, or array that you created earlier. (4 points)

```
[36]: observed_statistic_ab = sampled_ages.group('Gender', np.mean).column('Age_
      ↪mean').item(1) - sampled_ages.group('Gender', np.mean).column('Age mean').
      ↪item(0)
      observed_statistic_ab
```

[36]: 1.314102564102562

Question 1.7. Assign `shuffled_labels` to an array of shuffled male/female labels. The rest of the code puts the array in a table along with the data in `sampled_ages`. (4 points)

```
[37]: shuffled_labels = sampled_ages.sample(with_replacement=False).column('Gender')
      original_with_shuffled_labels = sampled_ages.with_columns('Shuffled Label',
      ↪shuffled_labels)
      original_with_shuffled_labels
```

```
[37]: Gender | Age | Shuffled Label
      male | 23 | male
      male | 29 | male
      male | 29 | female
      female | 49 | female
      female | 33 | female
      male | 31 | female
      male | 60 | female
      male | 38 | female
      female | 60 | male
      female | 27 | male
      ... (490 rows omitted)
```

Question 1.8. The comparison below uses the array `shuffled_labels` from Question 3.7 and the count `num_females` from Question 3.1.

For this comparison, assign the correct letter as a string (e.g. `correct_q8 = 'A'`) from one of the following options to the variable `correct_q8`. **Pretend this is a midterm problem and do not solve it using a code cell.** (4 points)

```
comp = np.count_nonzero(shuffled_labels == 'female') == num_females
```

A. `comp` is set to True. B. `comp` is set to False. C. `comp` is set to True or False, depending on how the shuffle came out.

```
[38]: correct_q8 = 'A'
      correct_q8
```

[38]: 'A'

Question 1.9. Define a function `simulate_one_statistic` that takes no arguments and returns one simulated value of the test statistic. We've given you a skeleton, but feel free to approach this question in a way that makes sense to you. Use as many lines of code as you need. Refer to the code you have previously written in this problem, as you might be able to re-use some of it. (4 points)

```
[39]: def simulate_one_statistic():
    "Returns one value of our simulated test statistic"
    shuffled_labels = sampled_ages.sample(with_replacement=False).
    ↪column('Gender')
    shuffled_tbl = sampled_ages.with_columns('Shuffled Label', shuffled_labels)
    group_means = shuffled_tbl.group('Shuffled Label', np.mean).column('Age_
    ↪mean')
    return group_means.item(1) - group_means.item(0)
```

After you have defined your function, run the following cell a few times to see how the statistic varies.

```
[40]: simulate_one_statistic()
```

```
[40]: 0.2964743589743577
```

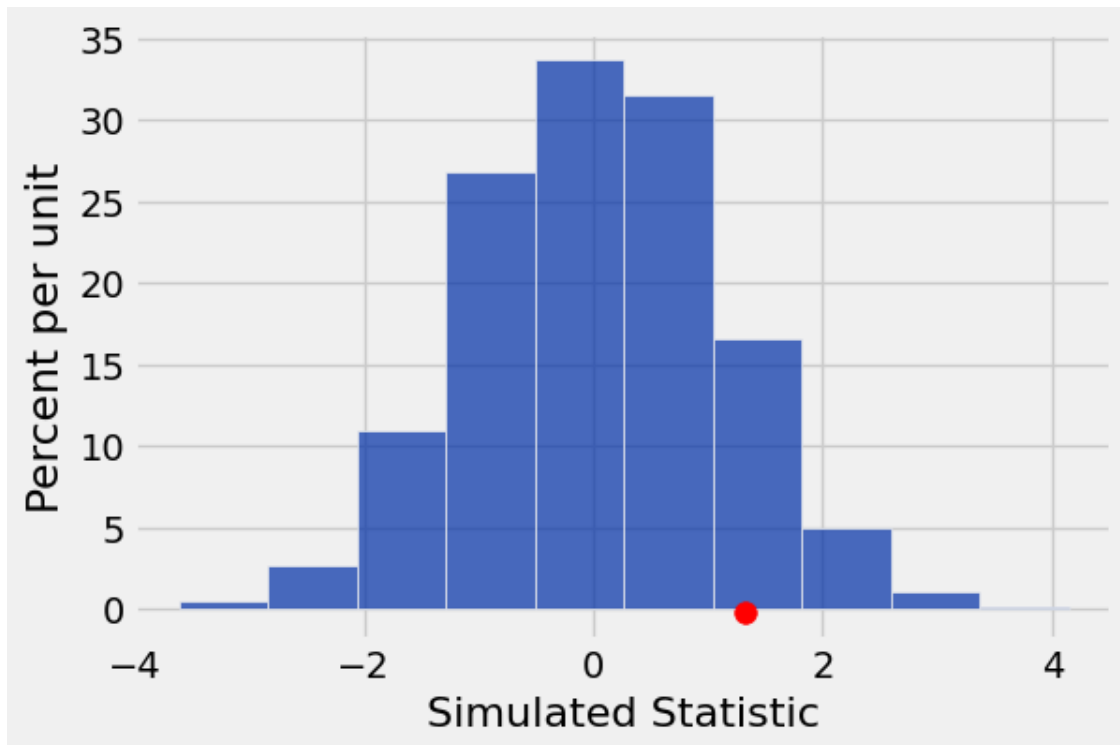
Question 1.10. Complete the cell to simulate 5,000 values of the statistic. We have included the code that draws the empirical distribution of the statistic and shows the value of `observed_statistic_ab` from Question 3.6. Feel free to use as many lines of code as you need. (4 points)

Note: This cell will take around a minute to run.

```
[41]: simulated_statistics_ab = make_array()

for i in np.arange(5000):
    simulated_statistics_ab = np.append(simulated_statistics_ab,
    ↪simulate_one_statistic())

# Do not change these lines
Table().with_columns('Simulated Statistic', simulated_statistics_ab).hist()
plt.scatter(observed_statistic_ab, -0.002, color='red', s=70);
```



Question 1.11. Use the simulation to find an empirical approximation to the p-value. Assign `p_val` to the appropriate p-value from this simulation. Then, assign `conclusion` to either `null_hyp` or `alt_hyp`. (4 points)

Note: Assume that we use the 5% cutoff for the p-value.

```
[42]: # These are variables provided for you to use.
null_hyp = 'The data are consistent with the null hypothesis.'
alt_hyp = 'The data support the alternative more than the null.'

p_val = np.count_nonzero(simulated_statistics_ab >= observed_statistic_ab) / 5000

if p_val <= 0.05:
    conclusion = alt_hyp
else:
    conclusion = null_hyp

p_val, conclusion # Do not change this line
```

```
[42]: (0.1208, 'The data are consistent with the null hypothesis.')
```


1.2 Submission

Make sure you have run all cells in your notebook in order before running the cell below, so that all images/graphs appear in the output. The cell below will generate a pdf file for you to submit.

Please save before exporting!

```
[44]: # should change the directory and file name matching to yours
!jupyter nbconvert './DS/lab15(SSU).ipynb' --to pdf
```

```
[NbConvertApp] Converting notebook ./DS/lab15(SSU).ipynb to pdf
[NbConvertApp] Writing 50145 bytes to notebook.tex
[NbConvertApp] Building PDF
[NbConvertApp] Running xelatex 3 times: ['xelatex', 'notebook.tex', '-quiet']
[NbConvertApp] Running bibtex 1 time: ['bibtex', 'notebook']
[NbConvertApp] WARNING | bibtex had problems, most likely because there were no
citations
[NbConvertApp] PDF successfully created
[NbConvertApp] Writing 65646 bytes to DS/lab15(SSU).pdf
```